

Few-Step MeanFlow Residuals for Convection-Allowing Forecasting over Taiwan

研究生: 鍾鎮嶸
指導教授: 周承復

Outline

- Introduction
- Related work
- Dataset
- Method
- Experiments
- Discussion
- Conclusion

Outline

- Introduction
- Related work
- Dataset
- Method
- Experiments
- Discussion
- Conclusion

Introduction

- Hourly regional convection-allowing generative forecasting model (StormCast) is computationally expensive.
 - 35 network calls per forecast steps $\rightarrow$ a 50-member, 24-h ensemble $\approx$ 42000 U-Net passes.
- Precipitation is hard to predict yet on generative-based forecasting models.

Motivation

- A sharp hourly regional convection-allowing generative forecasting model
 - Time resolution: 1 hour
 - Spatial resolution: ~2 km
 - Inference at 1-2 steps
 - Achieve similar or better forecasting quality

Outline

- Introduction
- Related work
- Dataset
- Method
- Experiments
- Discussion
- Conclusion

Numerical Weather Prediction

- Solve the physics on a grid (IFS, GFS, WRF/RWRF). Accurate, physically grounded, expensive.

Resolutions and the CFL condition

- Spatial resolution
 - grid spacing / pixel resolution of weather data
 - Two regimes:
 - Synoptic / Global (~10-25 km): resolves cyclones
 - Convection-allowing / regional (~1-4 km): resolves thunderstorms, orographic precipitation
- Temporal resolution
 - time step between two adjacent weather data
- Changes in resolutions has to obey the Courant–Friedrichs–Lewy condition to be mathematically stable

$$C = \left(\frac{|u|}{\Delta x} + \frac{|v|}{\Delta y} + \frac{|w|}{\Delta z} \right) \Delta t \leq C_{\max} \quad \left(C_{\max} \sim 1 \text{ for explicit schemes} \right) \Delta x \downarrow 2\times \implies \Delta t \downarrow 2\times \implies \text{3-D cost} \uparrow 2^4 = 16\times$$

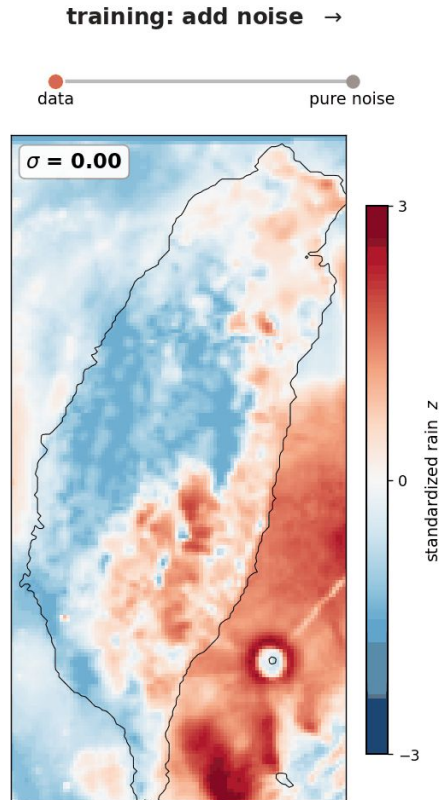
Numerical Weather Prediction's computational requirement

- ECMWF (European Centre for Medium-Range Weather Forecasts)
 - Global operational weather forecasting and massive climate re-analysis (ERA5)
 - CPU: 1,015,808 AMD EPYC Rome CPU cores
 - Memory: 2,050 Tb
 - 30.0 PetaFLOPS (10^{15})
- RIKEN Center for Computational Science
 - "Mega-ensemble" forecasting. (e.g., Running 1,000 simultaneous ensemble members to accurately predict probabilistic events like the July 2020 Kyushu heavy rain)
 - CPU: 7,630,848 Fujitsu A64FX cores
 - Memory: 4,850 Tb
 - 997 PetaFLOPS (10^{15})

The mesoscale bottleneck

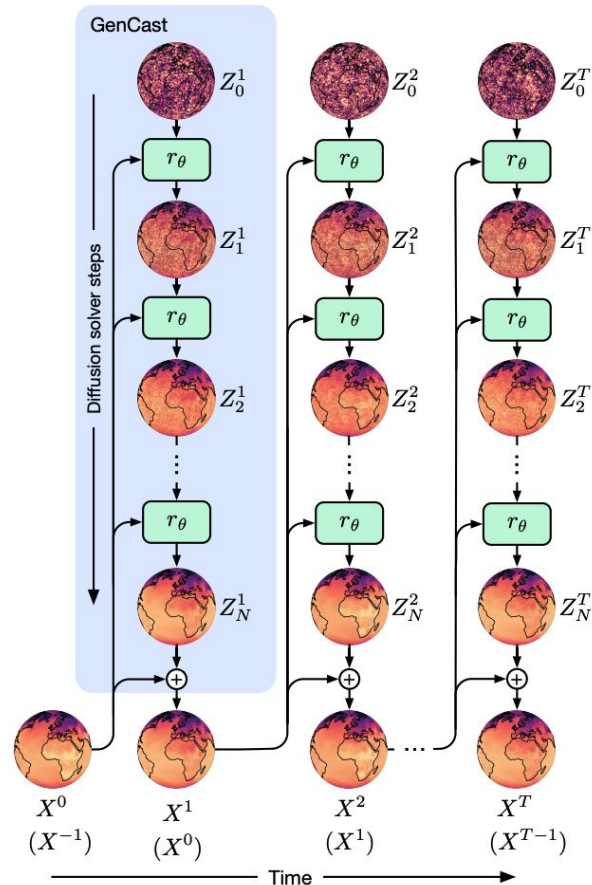
- Synoptic scale ($\sim 0.25^\circ$ grid, GFS / ECMWF-IFS): cheap, runs every 6 h.
- Convection-allowing models / CAMs (~ 3 km grid, HRRR / RWRF): must resolve thunderstorms, cold pools, gravity waves
 - $25/3 = 8.333$ times of change in resolution in each dimension
 - To obey CFL, computation is 8.333^3 (3 dimensions) * 8.333 (temporal change) = ~ 4821 more expensive

Diffusion model

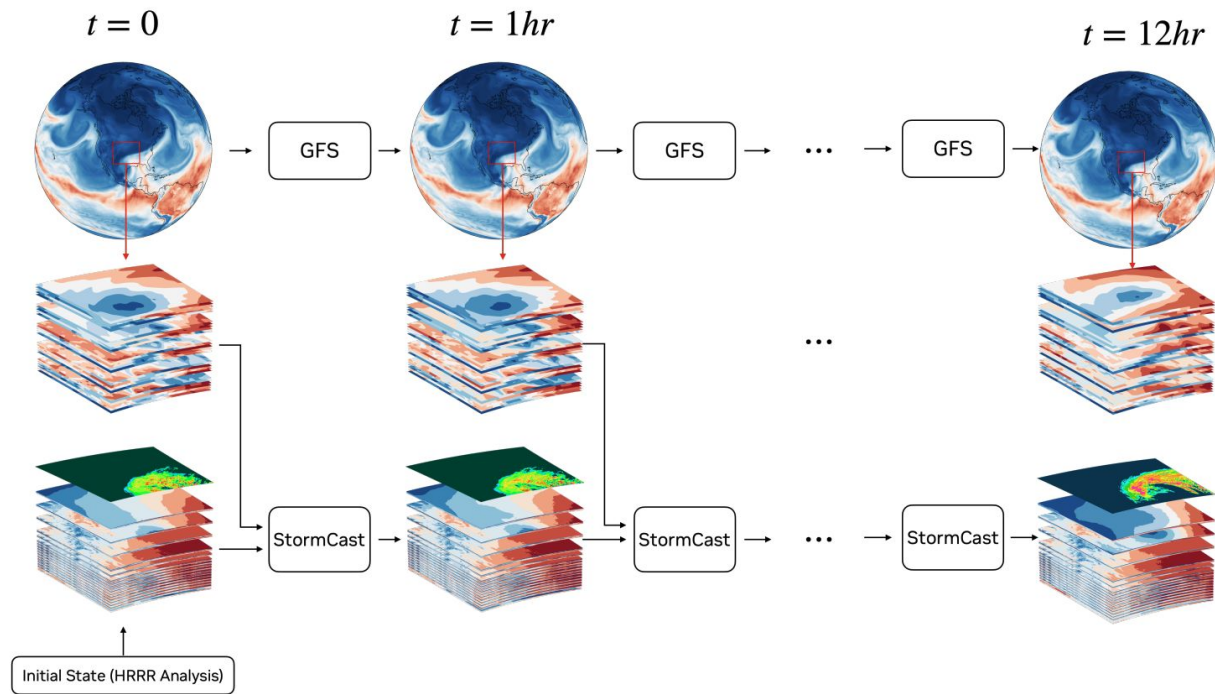


Google Deepmind's GenCast

- 15-day ensemble
- Spatial resolution: 0.25° (ERA5)
- Time resolution: 12 hours
- Domain: Global



Nvidia's StormCast

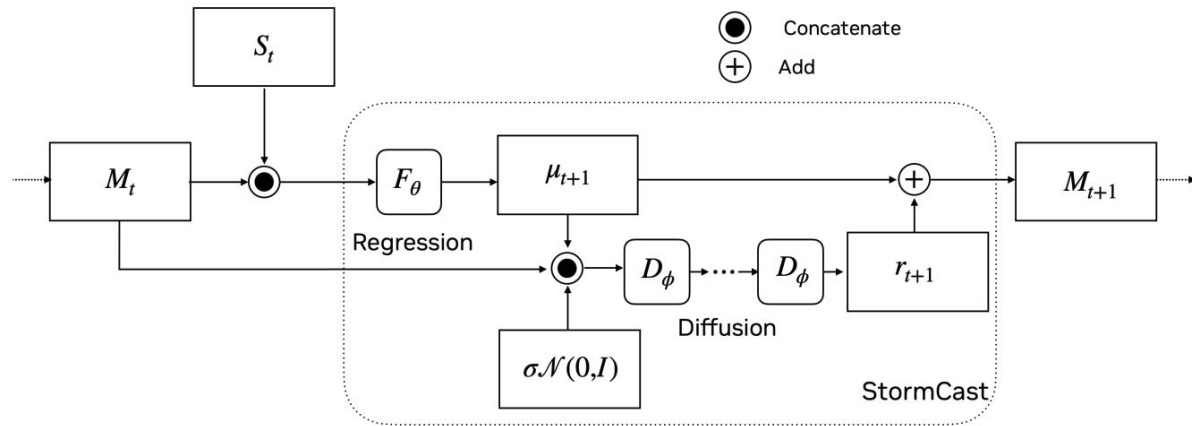


source: [Pathak et al. \(2024\), arXiv:2408.10958](https://arxiv.org/abs/2408.10958)

Nvidia's StormCast

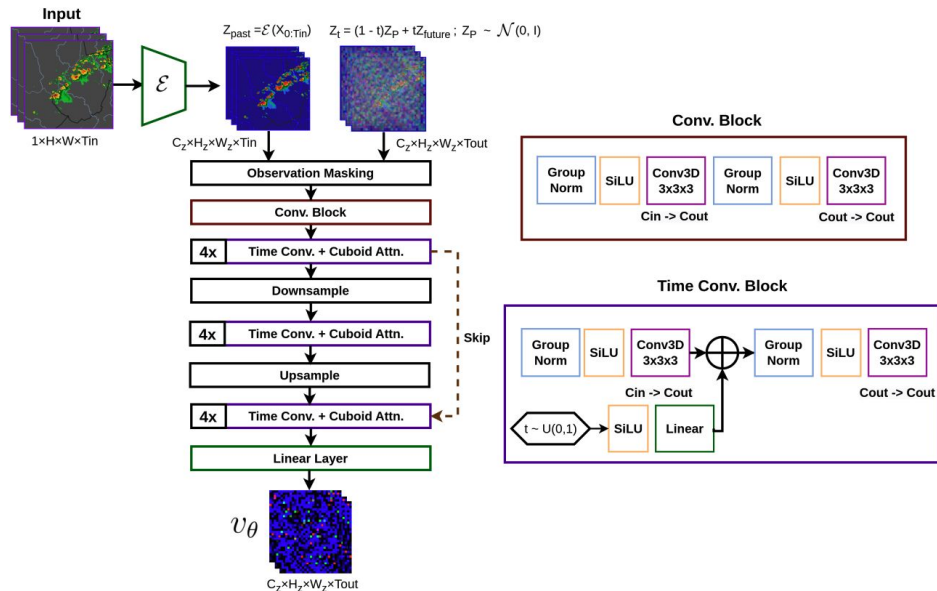
- 6-hour roll out
- 5-member ensemble
- Spatial resolution: 3 km
- Time resolution: 1 hour
- Coverage: Central US
- Domain:

1536 km × 1920 km



FlowCast

- Non-autoregressive
- Spatial resolution: 1 km
- Time resolution: 5 minutes
- Coverage: US and Solvenian
- Ingests: 13 past frames
- Predicts: 12 futures frames



Outline

- Introduction
- Related work
- Dataset
- Method
- Experiments
- Discussion
- Conclusion

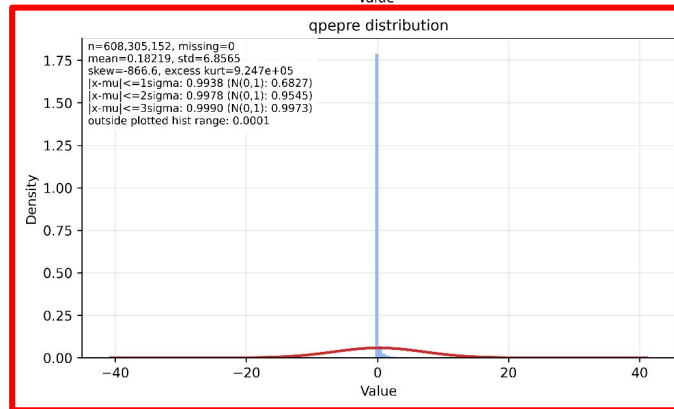
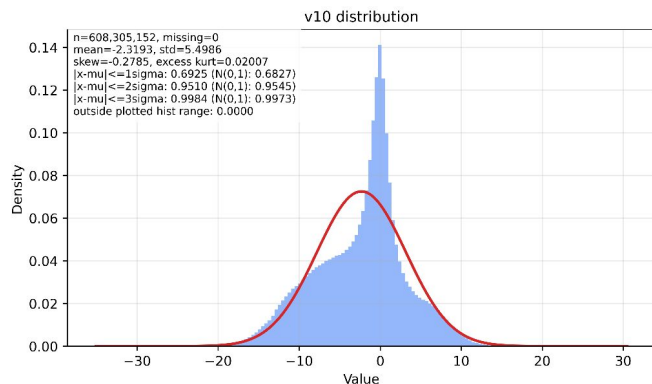
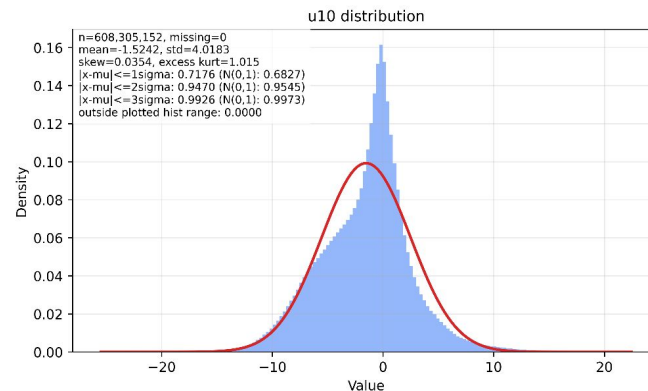
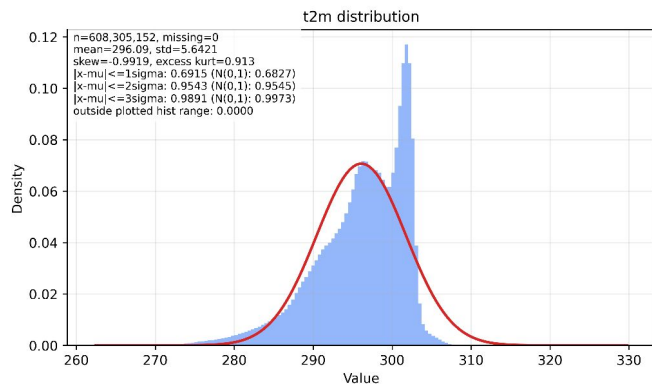
Dataset: ERA5, RWRF, Qpepre

- Latitude: 21.6° - 25.6° N
- Longitude: 119.75° - 122.25° E
- Time resolution: 1 hour
- Window: 2019/08 to 2022/12
- Samples: 29976 samples

	ERA5	RWRF	QPEPRE
Type	Reanalysis	Numerical model	Radar observation
Resolution	≈ 25 km	≈ 2 km	≈ 2 km
Role	LowRes input S_t	HighRes target X_t	HighRes target X_t
Channels used	24 (4 sfc + 5×4 levels)	3 (t2m, u10, v10)	1 (qpepre)
Native format	NetCDF	NetCDF (wrfout)	ASCII text

Code	Variable	Units	Levels (hPa)	# ch.
<i>Surface (single level)</i>				
mslp	Mean sea-level pressure	Pa	surface	1
t2m	2 m temperature	K	surface	1
u10	10 m zonal wind	m s^{-1}	surface	1
v10	10 m meridional wind	m s^{-1}	surface	1
<i>Upper-air (one channel per pressure level)</i>				
q	Specific humidity	kg kg^{-1}	1000, 850, 500, 250	4
t	Temperature	K	1000, 850, 500, 250	4
u	Zonal wind	m s^{-1}	1000, 850, 500, 250	4
v	Meridional wind	m s^{-1}	1000, 850, 500, 250	4
z	Geopotential	$\text{m}^2 \text{s}^{-2}$	1000, 850, 500, 250	4
Total				24

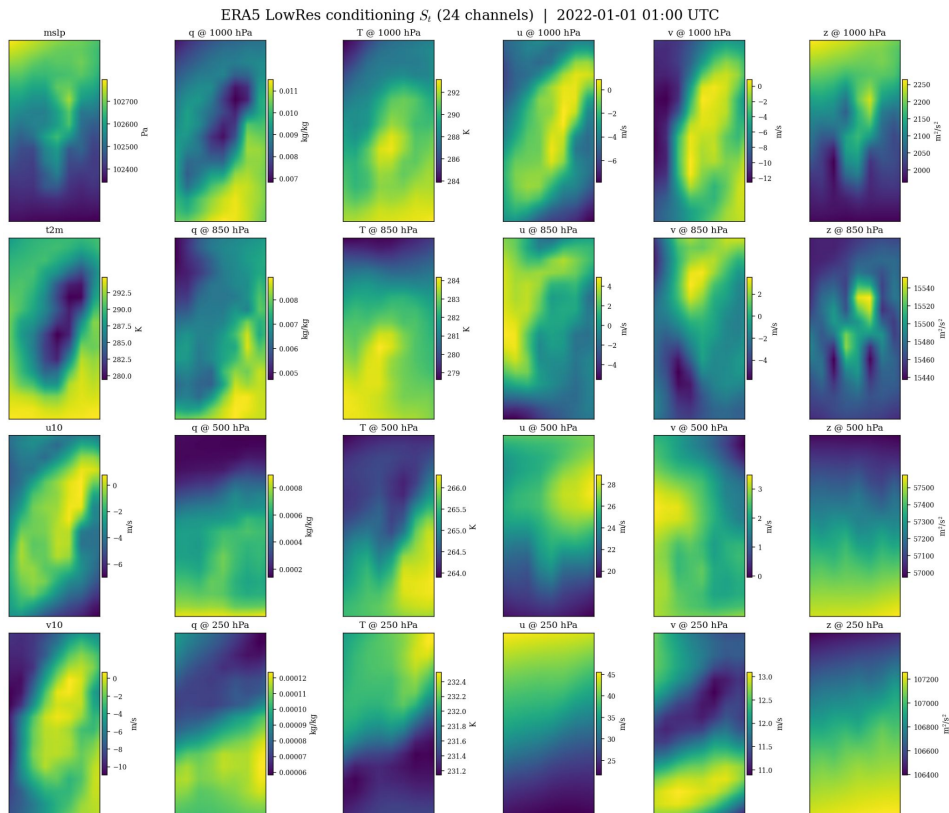
RWRF & Qpepre distributions



Log1p on Qpepre distributions

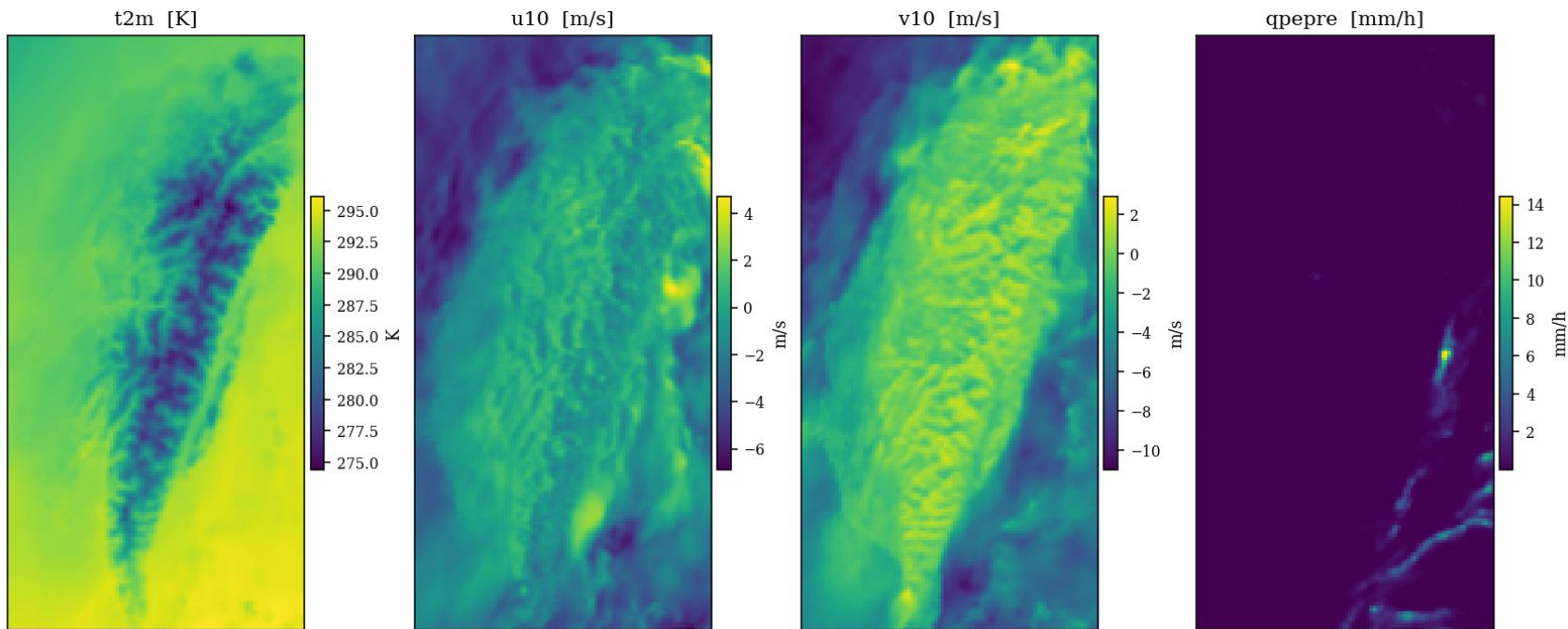
- MSE loss cares about absolute difference
- Predicting 0.1 mm/h instead of 1 mm/h (10× error) contributes the same $0.81 \text{ mm}^2/\text{h}^2$ to the loss as predicting 100 vs 100.9 mm/h (0.9% error). The model is told both mistakes are equally bad.
- log1p makes the loss measure log-ratios ($\log_{1p}(1) - \log_{1p}(0.1) \approx 0.60$ vs $\log_{1p}(100.9) - \log_{1p}(100) \approx 0.009$), so the gradient is concentrated where the physics actually differ.
- Perform $\log_{1p}$, $\ln(1 + x)$ where x is qpepre before training.
- Inverse with $e^p - 1$ for output, where p is transformed qpepre.

Dataset: ERA5



Dataset: RWRF and Qpepre

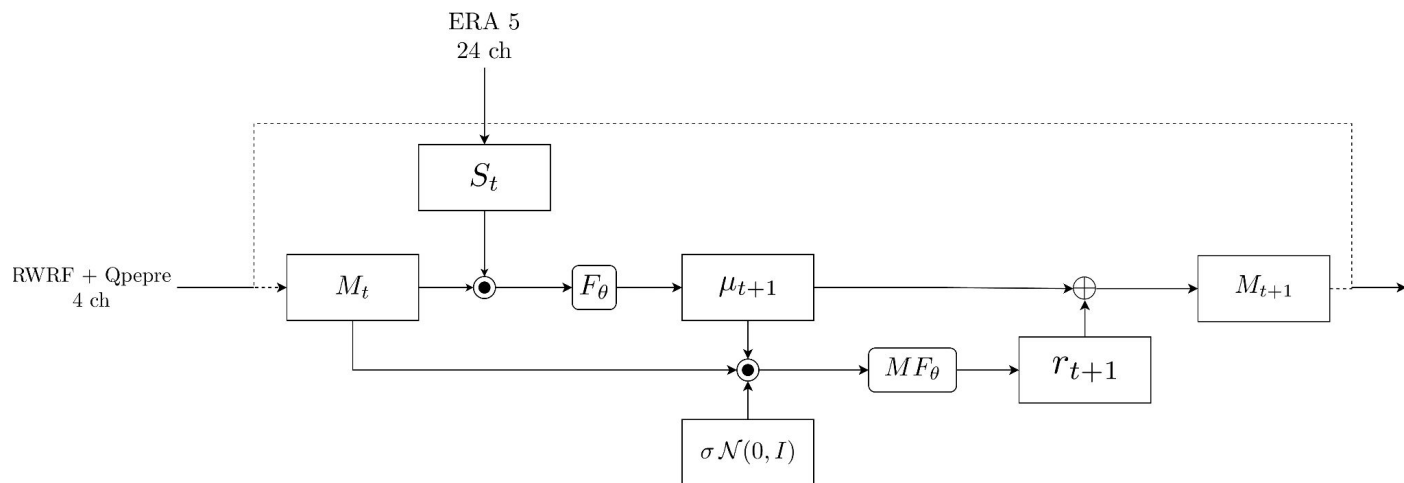
HighRes target X_{t+1} (4 channels) | 2022-01-01 01:00 UTC



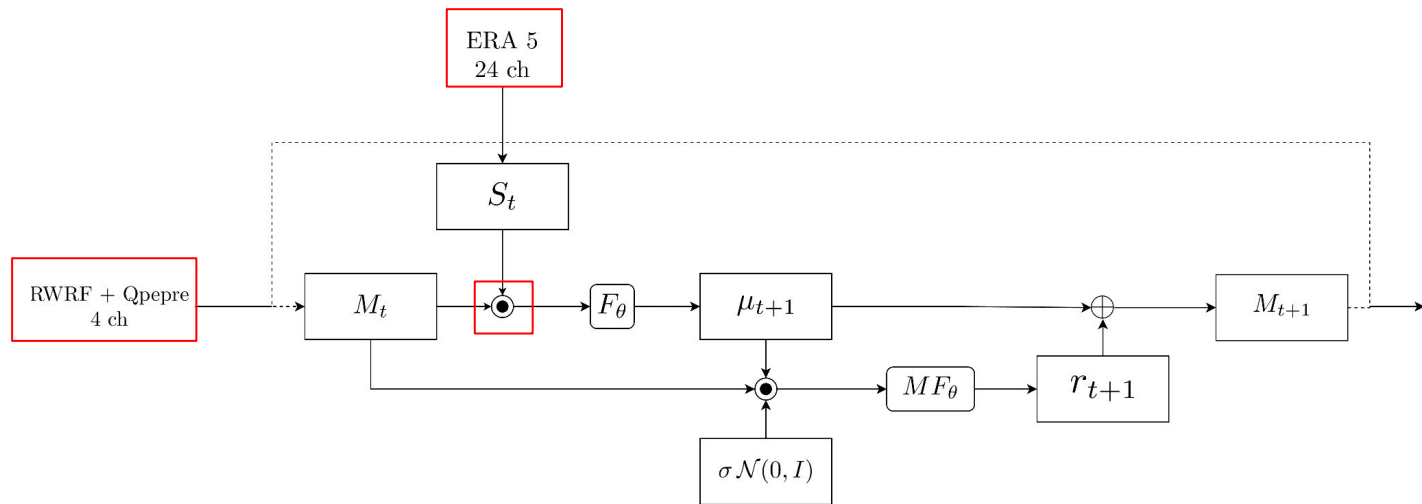
Outline

- Introduction
- Related work
- Dataset
- **Method**
- Experiments
- Discussion
- Conclusion

Method: Overview

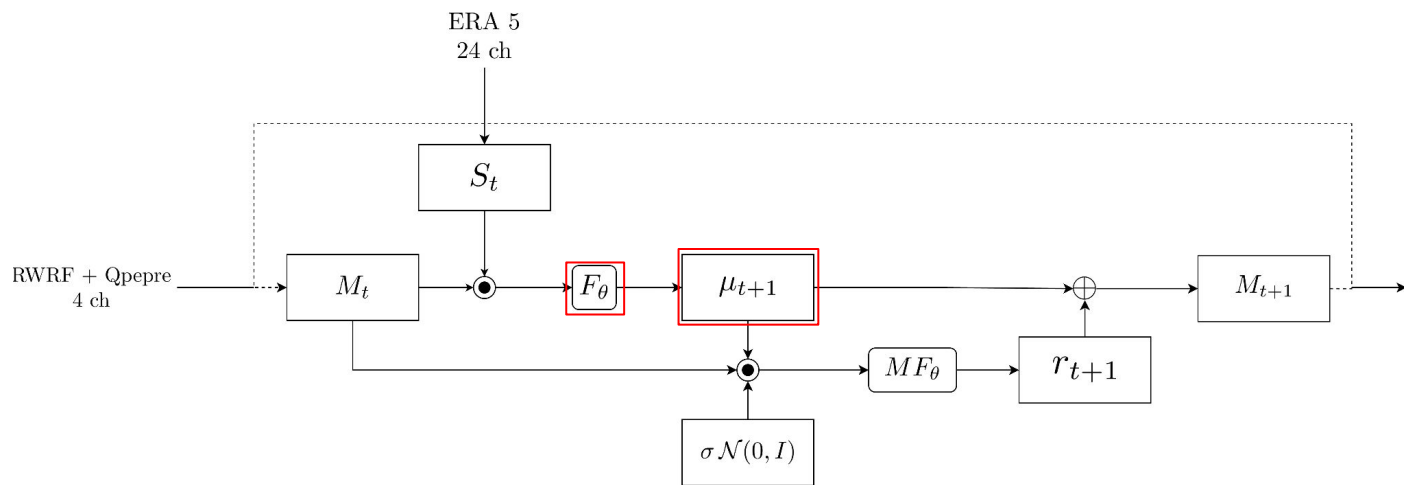


Method: Input and preprocessing



- Low res and high res data regridded into desired domain
- Concatenated as a tensor

Method: Regression model

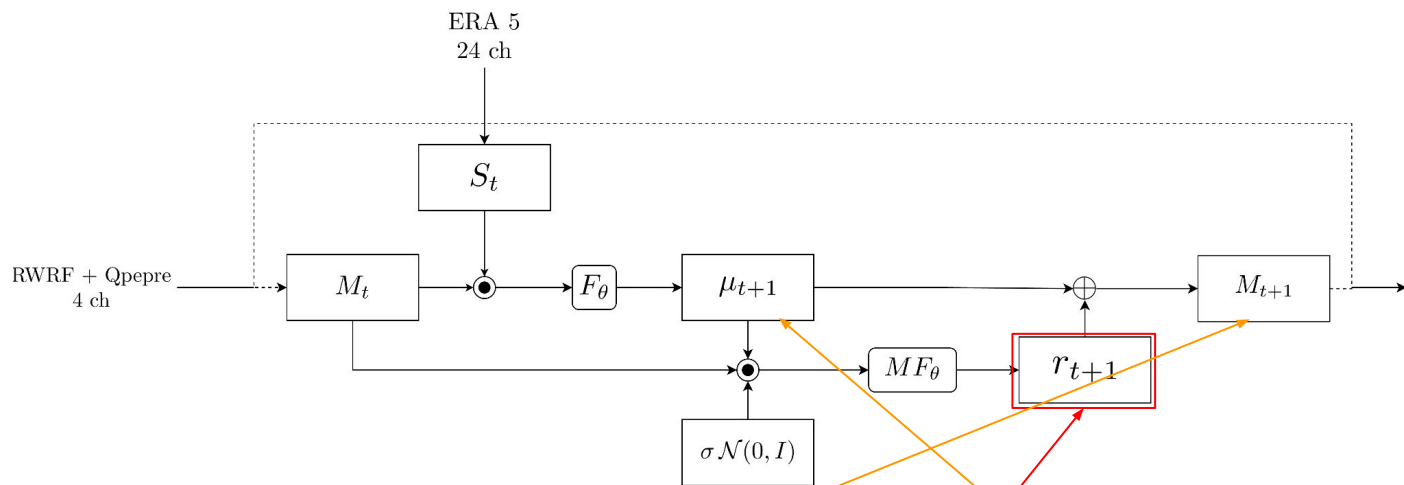


$$F_\theta(M_t, S_t) = \mathbb{E}[M_{t+1} \mid M_t, S_t]$$

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \|M_{t_{i+1}} - F_\theta(M_{t_i}, S_{t_i})\|_2^2$$

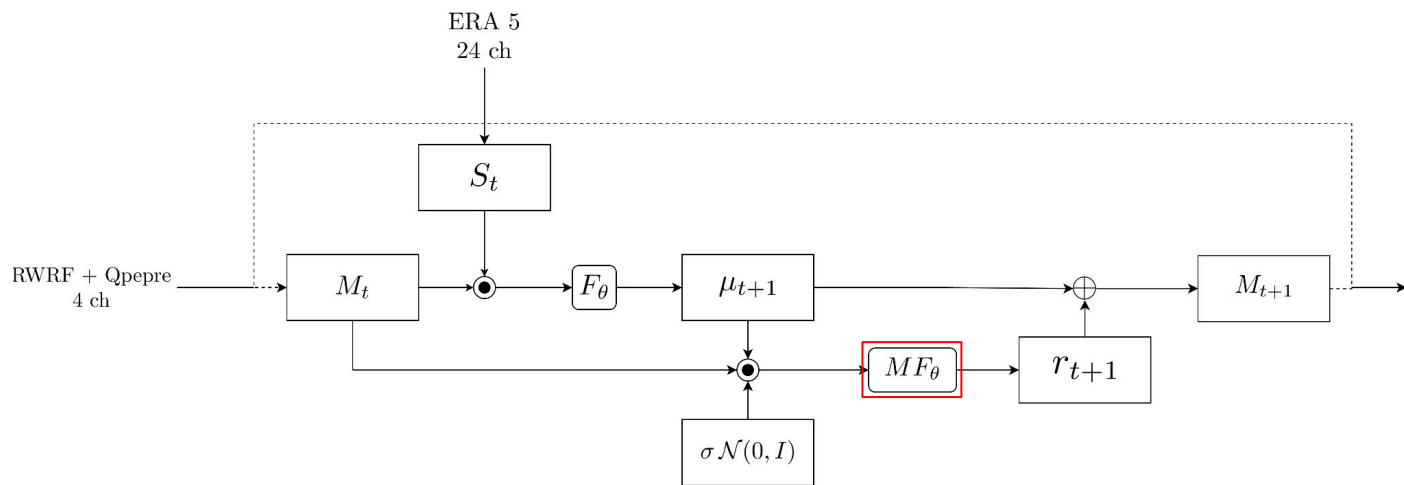
- Train a U-Net learn a deterministic regression F_θ to estimate the conditional mean.

Method: Residual



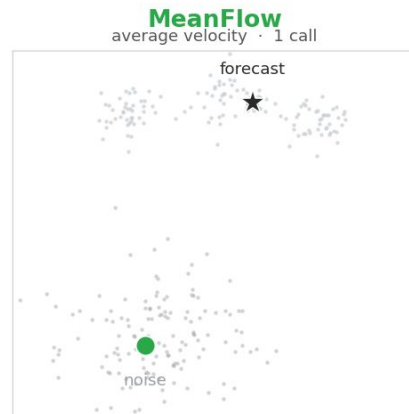
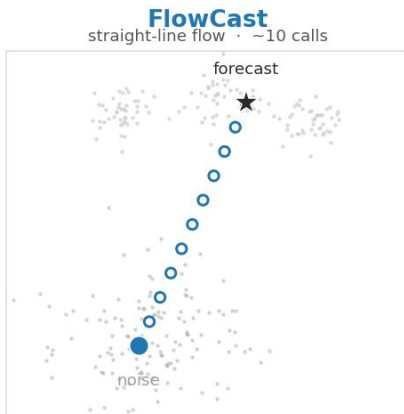
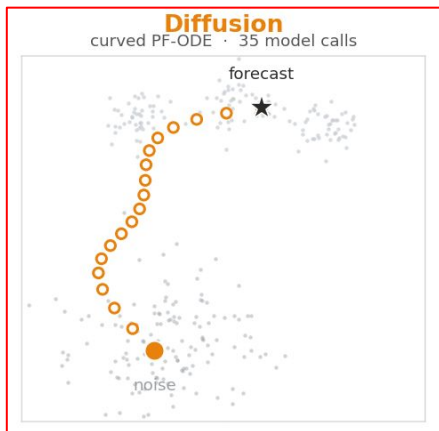
$$p(M_{t+1} | M_t, S_t) = r_{t+1} + \mu_{t+1}$$

Method: MeanFlow



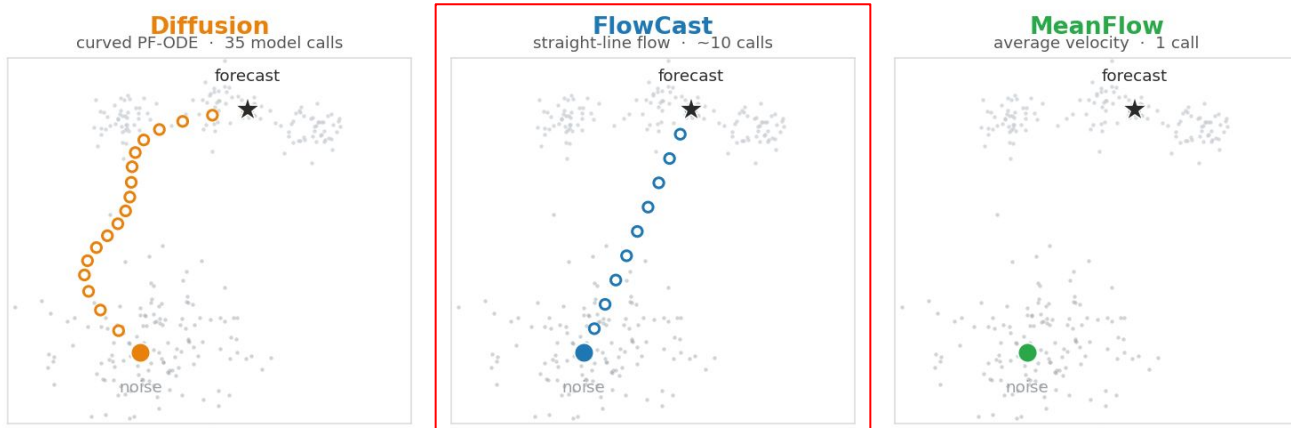
- Train a generative model to learn residual of next timestep

Method: Diffusion (Original Stormcast)



- StormCast uses EDM, an improved DDPM through better noise scheduling

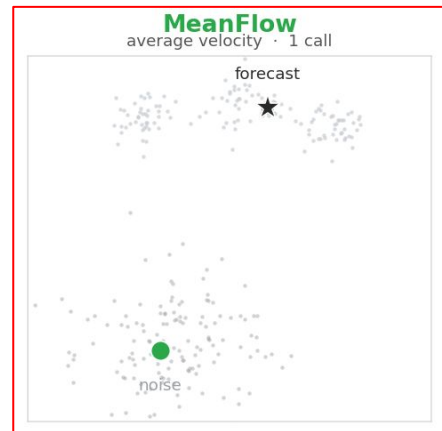
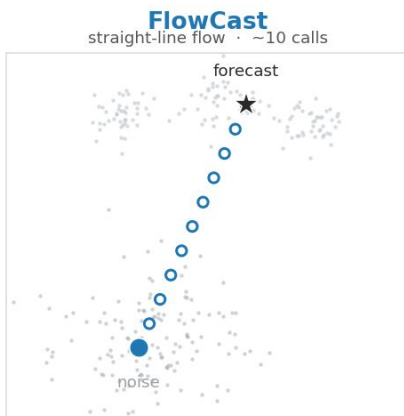
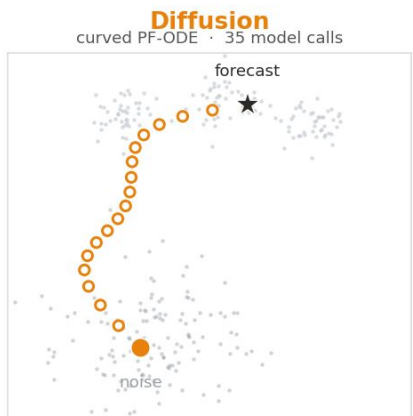
Method: MeanFlow



$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}(0,1) \\ (x_0, x_1) \sim q \\ x \sim p_t(x | x_0, x_1)}} \left[\left\| v_\theta(t, x) - u_t(x | x_0, x_1) \right\|_2^2 \right]$$

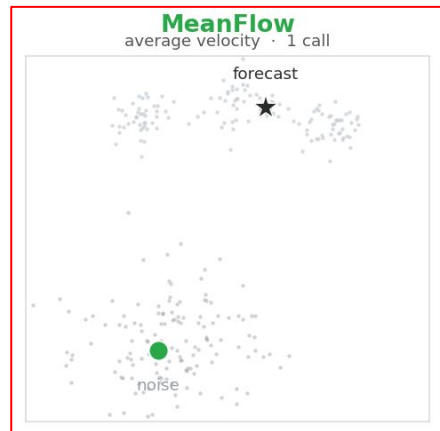
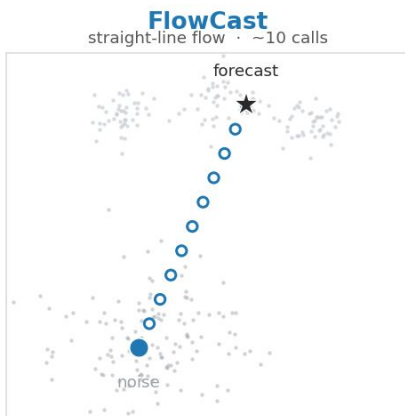
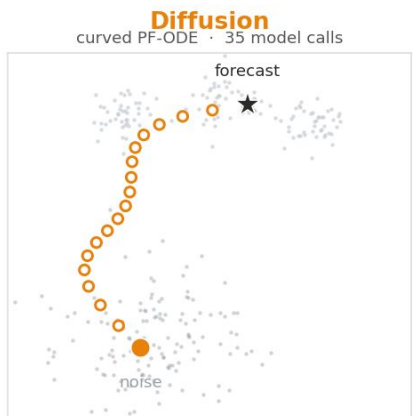
- Conditional flow matching learns the instantaneous velocity at any point between 0 to 1 to go from pure noise to target distribution (data)

Method: MeanFlow



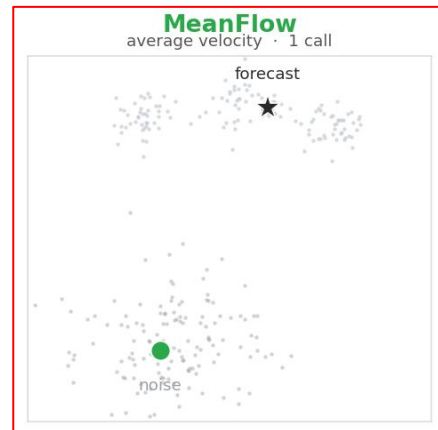
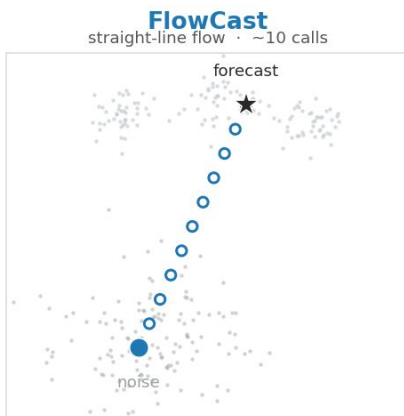
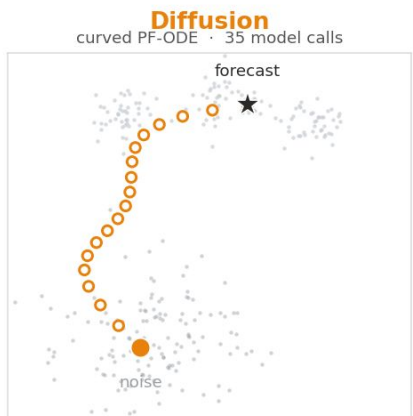
$$u(z_r, r, t) \triangleq \frac{1}{t-r} \int_r^t v(z_\tau, \tau) d\tau,$$

Method: MeanFlow



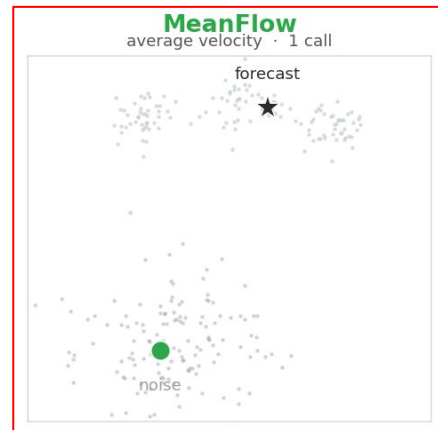
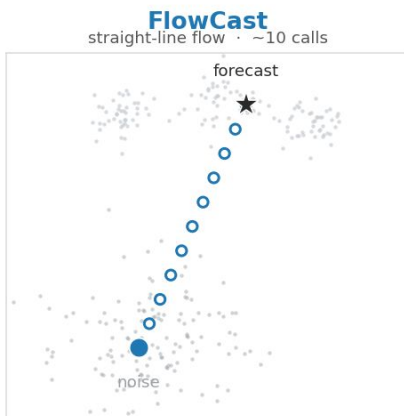
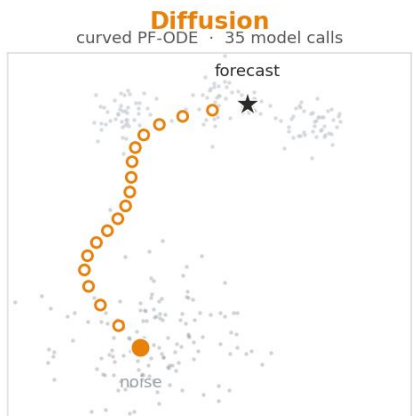
$$z_t = z_r + (t - r) u(z_r, r, t),$$

Method: MeanFlow



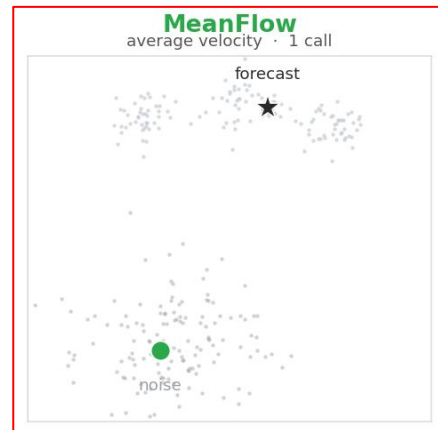
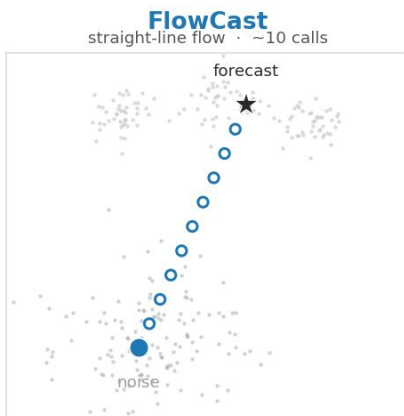
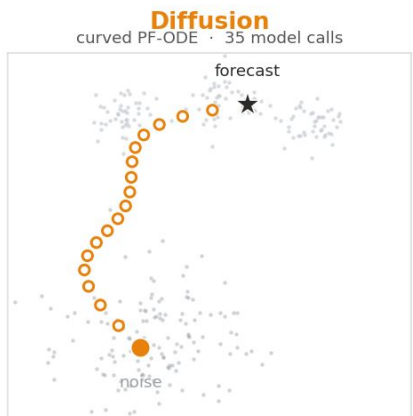
$$u(z_r, r, t) = v(z_r, r) + (t - r) \frac{d}{dr} u(z_r, r, t),$$

Method: MeanFlow



$$\frac{d}{dr} u(z_r, r, t) = v(z_r, r) \partial_z u + \partial_r u.$$

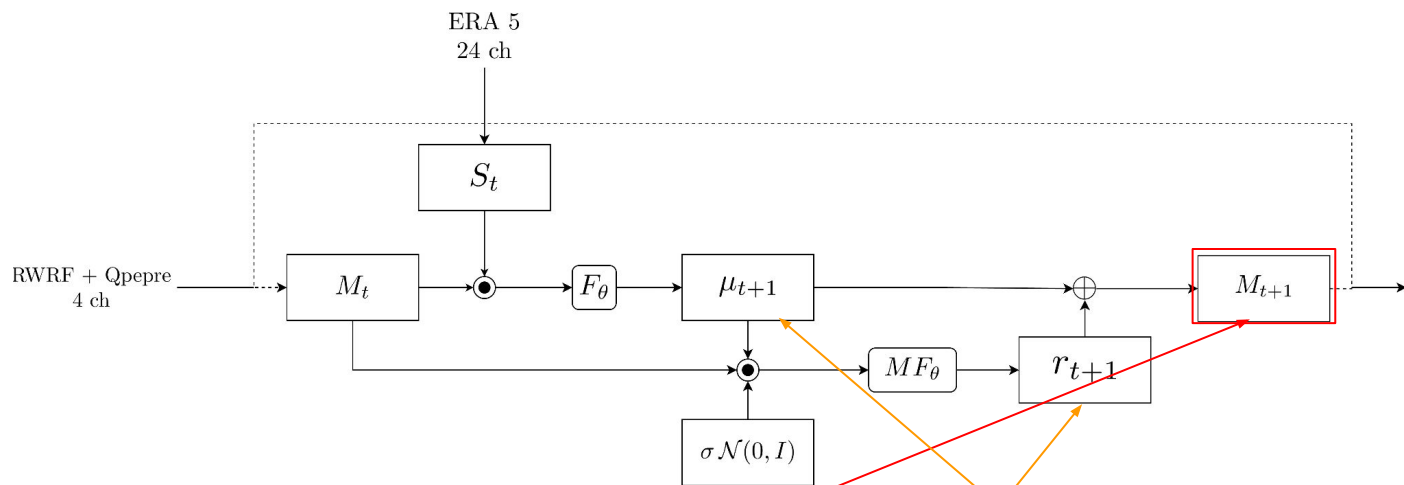
Method: MeanFlow



$$\mathcal{L}_{\text{MF}} = \mathbb{E} \left[\text{sg}(w) \left\| u_{\theta}(z_r, r, t, c_t) - \text{sg}(u_{\text{tgt}}) \right\|_{2, \beta}^2 \right] + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}},$$

$$u_{\text{tgt}} = v + (t - r)(v \partial_z u_{\theta} + \partial_r u_{\theta}),$$

Method: Next hour prediction



$$p(M_{t+1} | M_t, S_t) = r_{t+1} + \mu_{t+1}$$

Outline

- Introduction
- Related work
- Dataset
- Method
- Experiments
- Discussion
- Conclusion

Outline

- Introduction
- Related work
- Dataset
- Method
- Experiments
- Discussion
- Conclusion

Outline

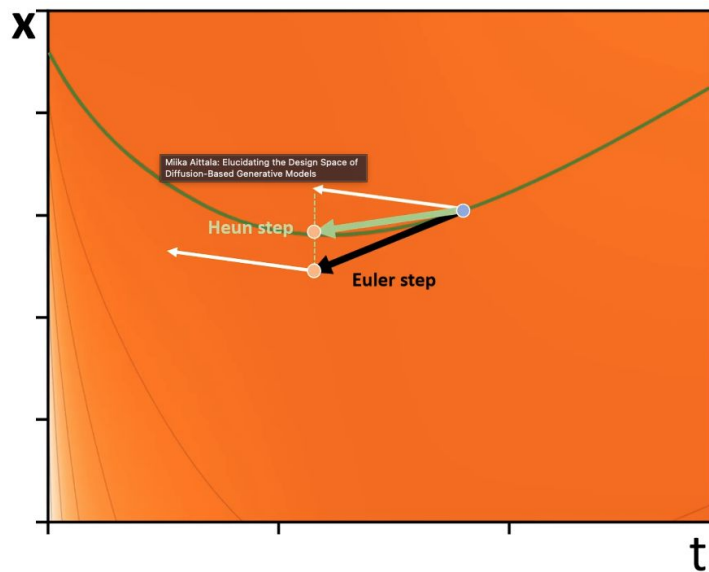
- Introduction
- Related work
- Dataset
- Method
- Experiments
- Discussion
- Conclusion

EDM

- Noise scheduling: Exactly how much noise is injected or removed at any given moment.
- Preconditioning: How the inputs and outputs are **scaled** so the neural network doesn't mathematically choke on extreme values.

EDM

- The Solver:



Higher-order solvers

- Clever sub-steps $\rightarrow$
higher accuracy,
higher cost
- Empirically, 2nd order **Heun method** strikes best balance

StormCast (Nvidia, 2024), collaboration with NCDR

